

ASSIGNMENT – 4

Simulation of Go-Back-N (GBN) ARQ Protocol Using Python (Reliable File Transfer in a Cloud Storage System)

AIM

To simulate the Go-Back-N (GBN) Automatic Repeat reQuest (ARQ) protocol using a TCP client-server application. A cloud storage service transfers files between a central server and multiple client devices; frames may be lost or corrupted during transmission. In GBN ARQ, the sender transmits multiple frames inside a sliding window without waiting for individual acknowledgements, and on a timeout retransmits the lost frame together with every outstanding frame in the current window.

The server accepts clients concurrently using Python threads and simulates the receiver: it receives frames in sequence, sends cumulative acknowledgements, and can be configured to simulate frame loss and acknowledgement loss. The client takes the number of frames and window size as input and displays frames transmitted, acknowledgements received, retransmitted frames, and the final transmission status. The performance is then analyzed for error-free transmission, single-frame loss, multiple-frame loss, acknowledgement loss, and different window sizes.

ALGORITHM

- CLIENT (sender): read number of frames and window size; send one window of frames; wait for cumulative ACKs with a 1 s timeout.
- CLIENT: on timeout, print the timed-out frame and retransmit the whole outstanding window from the current base; repeat until all frames are ACKed.
- CLIENT: send DONE with the total frame count and print frames sent, retransmissions, ACKs received, and transmission efficiency.
- SERVER (receiver): keep an expected sequence number; for an in-order frame send ACK n, unless the frame is configured to be lost (then print "Frame n Lost" and wait for its retransmission).
- SERVER: on retransmission of a lost frame print "Retransmitted Frame n Received"; discard out-of-order frames; on duplicates resend the cumulative ACK.
- SERVER: if the ACK for a frame is configured as lost, receive the frame but send no ACK so the sender times out.

SERVER CODE – gbn_server.py

```
import socket
import sys
import threading

# go-back-n receiver server. simulates frame loss and ack loss based on user input.
# usage: gbn_server.py <lost_frames_csv or -1> <lost_ack or -1>
```

```
lost_frames = []
lost_ack = -1
if len(sys.argv) >= 2 and sys.argv[1] != "-1":
    lost_frames = [int(x) for x in sys.argv[1].split(",")]
if len(sys.argv) >= 3 and sys.argv[2] != "-1":
    lost_ack = int(sys.argv[2])

print_lock = threading.Lock()

def log(msg):
    # lock so two threads never write into the same line
    with print_lock:
        print(msg)

def handle_client(conn, addr):
    log(f"Client Connected: {addr}")
    try:
        # line based protocol so multiple frames in one tcp chunk stay separate
        f = conn.makefile("r", encoding="utf-8")
        expected = 0          # next frame number we are waiting for
        n_frames = None
        received = set()      # frames accepted so far
        retrans_pending = -1  # frame whose retransmission we are waiting for
        while True:
            line = f.readline()
            if not line:
                break
            msg = line.strip()
            parts = msg.split()
            if parts[0] == "DONE":
                n_frames = int(parts[1])
                break
            if parts[0] == "FRAME":
                n = int(parts[1])
                if n == expected:
                    if n in lost_frames:
                        # frame never arrives, sender must retransmit
                        log(f"Frame {n} Lost")
                        lost_frames.remove(n)
                        retrans_pending = n
                    elif retrans_pending == n:
                        log(f"Retransmitted Frame {n} Received")
                        log(f"ACK {n} Sent")
                        conn.sendall(f"ACK {n}\n".encode())
                        expected += 1
```

```
        retrans_pending = -1
        received.add(n)
    elif n == lost_ack:
        # frame is received but the ack is lost on the way back
        log(f"Frame {n} Received")
        log(f"ACK {n} Lost")
        expected += 1
        received.add(n)
    else:
        log(f"Frame {n} Received")
        log(f"ACK {n} Sent")
        conn.sendall(f"ACK {n}\n".encode())
        expected += 1
        received.add(n)
    elif n in received:
        # duplicate: frame was already accepted, resend cumulative ack
        log(f"Duplicate Frame {n} Received")
        log(f"ACK {n} Sent")
        conn.sendall(f"ACK {n}\n".encode())
    else:
        # out of order frame while the lost one is still missing
        log(f"Discarding Frame {n}")
    if n_frames is not None and expected == n_frames:
        log("All Frames Received Successfully")
    else:
        log("Connection closed before all frames arrived")
except ConnectionResetError:
    pass
conn.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(("127.0.0.1", 5006))
server.listen(5)
print(f"Go-Back-N Receiver Server Started (lost frames: {lost_frames}, lost ack: {lost_ack})")
print("Server listening on 127.0.0.1:5006")

while True:
    conn, addr = server.accept()
    # one thread per client so multiple senders are served concurrently
    t = threading.Thread(target=handle_client, args=(conn, addr), daemon=True)
    t.start()
```

CLIENT CODE - gbn_client.py

```
import socket
import sys

# go-back-n sender client. sends frames inside a sliding window and retransmits
# the whole outstanding window when a timeout happens.
# usage: gbn_client.py <number_of_frames> <window_size>

n_frames = int(sys.argv[1])
window = int(sys.argv[2])

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5006))
client.settimeout(1.0)

base = 0
total_sent = 0
retransmissions = 0
acks = 0
buf = ""

def read_line():
    # read one newline terminated line, returns None on timeout or close
    global buf
    while "\n" not in buf:
        try:
            chunk = client.recv(1024).decode()
        except socket.timeout:
            return None
        if not chunk:
            return None
        buf += chunk
    line, buf = buf.split("\n", 1)
    return line.strip()

def send_frames(frames, announce=True):
    # send one window of frames
    global total_sent
    if announce:
        if len(frames) == 1:
            print(f"Sending Frame: {frames[0]}")
        else:
            print("Sending Frames:", " ".join(map(str, frames)))
    for frame in frames:
        client.sendall(f"FRAME {frame}\n".encode())
        total_sent += 1
```

```
def wait_for_acks(end):
    # receive acks until the whole window is acked, returns False on timeout
    global base, acks
    while base < end:
        resp = read_line()
        if resp is None:
            return False
        if resp.startswith("ACK"):
            k = int(resp.split()[1])
            print(f"ACK Received: {k}")
            acks += 1
            if k == base:
                base += 1
    return True

while base < n_frames:
    end = min(base + window, n_frames)
    send_frames(list(range(base, end)))
    if wait_for_acks(end):
        continue
    # timeout: retransmit every outstanding frame from the current base
    print(f"Timeout for Frame {base}")
    retrans = list(range(base, min(base + window, n_frames)))
    print("Retransmitting Frames:", " ".join(map(str, retrans)))
    retransmissions += len(retrans)
    send_frames(retrans, announce=False)
    # keep waiting for the retransmitted window, retrying on repeated timeouts
    while not wait_for_acks(min(base + window, n_frames)):
        retrans = list(range(base, min(base + window, n_frames)))
        print(f"Timeout for Frame {base}")
        print("Retransmitting Frames:", " ".join(map(str, retrans)))
        retransmissions += len(retrans)
        send_frames(retrans, announce=False)

client.sendall(f"DONE {n_frames}\n".encode())
print("Transmission Completed Successfully")
print()
print(f"Frames Sent           : {total_sent}")
print(f"Frames Retransmitted   : {retransmissions}")
print(f"ACKs Received           : {acks}")
print(f"Transmission Efficiency : {n_frames / total_sent * 100:.2f}%")
client.close()
```

SERVER OUTPUT (TEST CASE 2 – SINGLE FRAME LOSS)

```
Go-Back-N Receiver Server Started (lost frames: [3], lost ack: -1)
Server listening on 127.0.0.1:5006
Client Connected: ('127.0.0.1', 61229)
Frame 0 Received
ACK 0 Sent
Frame 1 Received
ACK 1 Sent
Frame 2 Received
ACK 2 Sent
Frame 3 Lost
Retransmitted Frame 3 Received
ACK 3 Sent
Frame 4 Received
ACK 4 Sent
Frame 5 Received
ACK 5 Sent
Frame 6 Received
ACK 6 Sent
Frame 7 Received
ACK 7 Sent
All Frames Received Successfully
```

TEST CASE 1 – CLIENT OUTPUT (ERROR-FREE TRANSMISSION)

```
Sending Frames: 0 1 2 3
ACK Received: 0
ACK Received: 1
ACK Received: 2
ACK Received: 3
Sending Frames: 4 5 6 7
ACK Received: 4
ACK Received: 5
ACK Received: 6
ACK Received: 7
Transmission Completed Successfully

Frames Sent           : 8
Frames Retransmitted  : 0
ACKs Received         : 8
Transmission Efficiency : 100.00%
```

TEST CASE 2 – CLIENT OUTPUT (SINGLE FRAME LOSS)

```
Sending Frames: 0 1 2 3
ACK Received: 0
ACK Received: 1
ACK Received: 2
```

```
Timeout for Frame 3
Retransmitting Frames: 3 4 5 6
ACK Received: 3
ACK Received: 4
ACK Received: 5
ACK Received: 6
Sending Frame: 7
ACK Received: 7
Transmission Completed Successfully
```

```
Frames Sent           : 9
Frames Retransmitted  : 4
ACKs Received         : 8
Transmission Efficiency : 88.89%
```

TEST CASE 3 – CLIENT OUTPUT (ACKNOWLEDGEMENT LOSS)

```
Sending Frames: 0 1 2
ACK Received: 0
ACK Received: 1
Timeout for Frame 2
Retransmitting Frames: 2 3 4
ACK Received: 2
ACK Received: 3
ACK Received: 4
Sending Frame: 5
ACK Received: 5
Transmission Completed Successfully
```

```
Frames Sent           : 7
Frames Retransmitted  : 3
ACKs Received         : 6
Transmission Efficiency : 85.71%
```

TEST CASE 4 – CLIENT OUTPUT (MULTIPLE FRAME LOSS)

```
Sending Frames: 0 1 2 3
ACK Received: 0
ACK Received: 1
Timeout for Frame 2
Retransmitting Frames: 2 3 4 5
ACK Received: 2
ACK Received: 3
ACK Received: 4
ACK Received: 5
Sending Frames: 6 7 8 9
ACK Received: 6
```

Timeout for Frame 7
Retransmitting Frames: 7 8 9
ACK Received: 7
ACK Received: 8
ACK Received: 9
Transmission Completed Successfully

Frames Sent : 15
Frames Retransmitted : 7
ACKs Received : 10
Transmission Efficiency : 66.67%

TEST CASE 5 - CLIENT OUTPUT (WINDOW SIZE 2)

Sending Frames: 0 1
ACK Received: 0
ACK Received: 1
Sending Frames: 2 3
ACK Received: 2
ACK Received: 3
Sending Frames: 4 5
ACK Received: 4
Timeout for Frame 5
Retransmitting Frames: 5 6
ACK Received: 5
ACK Received: 6
Sending Frame: 7
ACK Received: 7
Transmission Completed Successfully

Frames Sent : 9
Frames Retransmitted : 2
ACKs Received : 8
Transmission Efficiency : 88.89%

PERFORMANCE COMPARISON

Test Case	Frames	Window	Error Simulated	Frames Sent	Retransmissions	ACKs	Efficiency	Observation
TC1	8	4	None	8	0	8	100.00%	No retransmission
TC2	8	4	Frame 3 lost	9	4	8	88.89%	Whole window 3-6 retransmitted

Test Case	Frames	Window	Error Simulated	Frames Sent	Retransmissions	ACKs	Efficiency	Observation
TC3	6	3	ACK 2 lost	7	3	6	85.71%	ACK loss causes timeout and retransmission
TC4	10	4	Frames 2 and 7 lost	15	7	10	66.67%	Two retransmission rounds
TC5	8	2	Frame 5 lost	9	2	8	88.89%	Smaller window, more windows and waiting

TASKS PERFORMED

Task	Details	Result
Threaded TCP server	Receiver accepts multiple sender clients on 127.0.0.1:5006 with one thread per connection	All five scenarios were served
Sliding window sender	Client sends one window of frames at a time and waits for cumulative ACKs	Windows of 4 and 2 frames transmitted
Frame loss simulation	Server configured with lost frame numbers (3; 2,7; 5)	Sender timed out and retransmitted the outstanding window
ACK loss simulation	Server received frame 2 but suppressed its ACK	Sender timed out and retransmitted frames 2-4
Duplicate handling	Receiver resends the cumulative ACK for duplicates	ACK 2 was recovered after the retransmission
Statistics	Client counted frames sent, retransmissions, ACKs and efficiency	Efficiency ranged from 66.67% to 100.00%

LEARNING OUTCOMES

- I implemented a sliding-window sender with cumulative ACKs and a 1 s retransmission timeout.
- I simulated frame loss, ACK loss, and out-of-order discarding at the receiver.
- I observed that one lost frame forces the retransmission of the whole window.
- I observed that smaller windows increase the number of windows but limit retransmission overhead.
- I computed transmission efficiency from real runs (100% error-free down to 66.67% with two losses).
- I learned that multiple frames in one TCP chunk must be separated with a line-based protocol.